Cover

GORM

ORM in Go for Beginers

Anuchit Chalothorn

Preface

Most Go developers meet SQL long before they meet an ORM, and the first ORM they meet tends to be the one that arrives with a tutorial. This book is for the moment after that: you have written a few queries by hand, you have started to repeat yourself, and you want to know whether GORM is worth putting between your structs and your database — and exactly what it will cost you if you do.

Who this book is for

This book is written for Go developers who are new to GORM, or new to ORMs in general. You do not need prior GORM experience. You do need to be comfortable with Go itself: structs, interfaces, error handling, and running a program from the command line. If you can read a function signature and reason about what it returns, you have enough to start.

You will get the most out of this book if you have a real project in mind — a service that stores users, orders, or events — because the chapters build toward the decisions that project will force on you. If you have never run a database at all, Chapter 2 walks through installing a driver and opening a connection from nothing.

This book is not a reference manual. GORM's full API surface is large, and the official documentation at https://gorm.io/docs/ is the right place to look up an individual method. What this book adds is the order: which concept you need before which other one, where the sharp edges are, and why the API behaves the way it does.

What you will learn

The book starts with the two things GORM is built on: declaring models, and the conventions that map your Go structs onto tables and columns. Once that foundation is in place, the four core operations — create, query, update, and delete — each get a chapter, because most bugs in day-to-day GORM code live in the details of one of them.

From there the book moves into the parts that separate working code from code you can maintain. You will model relationships between tables and learn how to load related data without falling into the N+1 trap. You will use migrations to evolve a schema without losing data, hooks to run logic at fixed points in the write lifecycle, and transactions to make several operations succeed or fail together. Later chapters cover the SQL builder for the cases the fluent API does not reach, scopes and sessions for keeping query logic reusable, and the type-safe generics API alongside the traditional one.

The closing chapters are about production: performance, observability, security, and testing against a

real database engine. By the end you should be able to take a service from an empty schema to something you would be willing to run, and to explain the trade-offs you made along the way.

How the examples are set up

The examples are written to be run, not just read. They target GORM v2, the version distributed as gorm.io/gorm, and the commands to install it appear at the start of Chapter 2:

```
go get -u gorm.io/gorm
go get -u gorm.io/driver/sqlite
```

Most examples use SQLite through gorm.io/driver/sqlite, because it runs in memory with no server to install and keeps the focus on the GORM code rather than on database administration. Where a behavior differs on PostgreSQL or MySQL — locking, upsert syntax, JSON handling — the text says so explicitly and shows the driver-specific form. Nothing in the core chapters depends on SQLite-only behavior.

Struct and field names follow GORM's own conventions, so a `User` struct becomes a `users` table and an `ID` field becomes the primary key. Where an example deliberately breaks a convention, it says why.

Code is kept as small as it can be while still being complete enough to run. When a snippet omits something — error handling around a `gorm.Open` call, say — it is because that code has already been established in an earlier chapter, not because it can be skipped in a real program.

One note on accuracy: GORM ships frequently, and behavior can change between releases. Every API in this book was checked against the official documentation, and where a feature depends on a particular version, that version is stated. When your own GORM version disagrees with this book, the documentation wins.

Table of Contents

Chapter 1: Introduction 1

What is an ORM 1

Raw SQL vs an ORM 4

What GORM provides 5

Traditional API and Generics API 8

What GORM does not do for you 11

Summary 13

Chapter 2: Setup 15

Installing GORM and a driver 15

Connecting to SQLite, MySQL, and PostgreSQL 20

gorm.Config and connection options 25

Connection pools and database/sql settings 30

A first working program 35

Summary 40

Chapter 1: Introduction

Hand-written SQL tends to accumulate. A query that was fine at first becomes one of forty scattered across a service, each a chance to misspell a column or forget a parameter placeholder. GORM removes that repetition by translating your Go structs into database operations. This chapter sets out what an ORM does, what GORM specifically offers, and where its responsibility stops and yours begins.

What is an ORM

An ORM — Object-Relational Mapping — maps your language's data structures onto the tables and columns of a relational database. In Go, your structs become tables, struct fields become columns, and queries you express in Go are translated into SQL and sent to the database.

The mapping runs in both directions. A struct value you pass in is written as a row; rows that come back are read into struct values, so you rarely copy a column into a field by hand.

This addresses the object-relational impedance mismatch: a struct is a set of named fields with Go types, while a table is a set of rows governed by SQL types, constraints, and keys. Bridging the two manually is tedious and easy to get wrong. An ORM does it once, systematically, in one place.

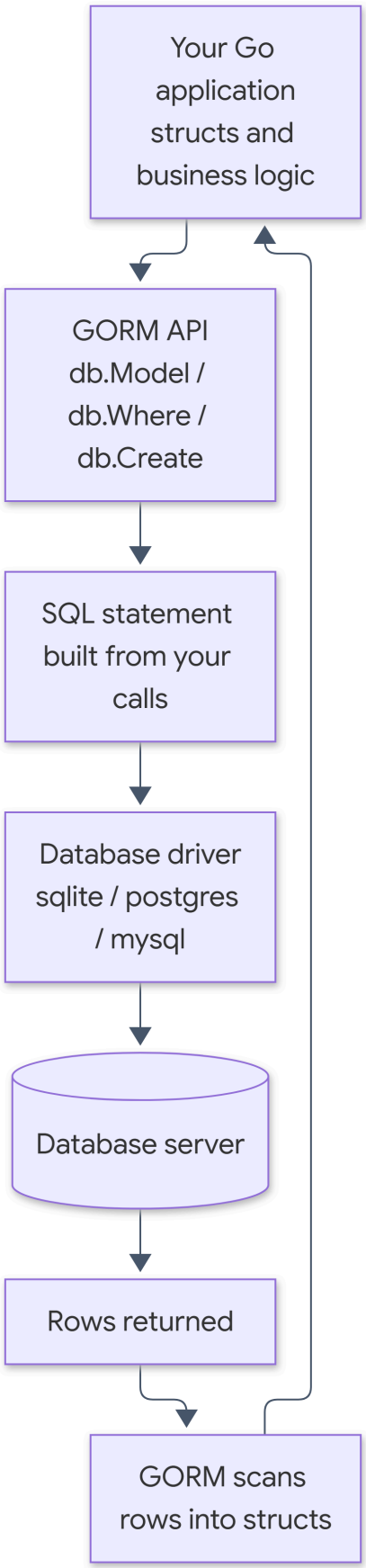

Your Go application structs and business logic
GORM API db.Model / db.Where / db.Create
SQL statement built from your calls
Database driver sqlite / postgres / mysql
Database server
Rows returned
GORM scans rows into structs

Fig 1.1 Where GORM sits between your Go application and the database server

The diagram places GORM as a layer between your application and the database server. Your code calls methods such as db.Model, db.Where, and db.Create; GORM builds a SQL statement from those calls and hands it to the configured driver, which sends it to the server. Rows return through the driver and GORM scans them into your structs. You deal in structs at both ends, while SQL and the wire protocol stay underneath.

Raw SQL vs an ORM

Raw SQL is a legitimate alternative, and for some systems it is the better one. Writing queries by hand with database/sql gives exact control over each statement, keeps performance characteristics easy to reason about, and never produces a query you did not intend.

The cost is repetition and drift. Every new struct means more insert, update, and select code; every join is another string to keep in sync with the schema. Rename a column

and the burden of finding every query that mentions it falls on you.

An ORM inverts that trade-off: you state what you want in terms of your Go types and the library generates the SQL. What you accept in return is indirection — the generated query is not always the one you would have written, and understanding it can mean inspecting what GORM actually produced.

This book takes the middle position it recommends: use the ORM for the routine bulk of data access, and drop to raw SQL where you need precise control. GORM supports both in one program, since it exposes the underlying connection and a SQL builder.

What GORM provides

GORM is a full-featured ORM for Go. Its feature set falls into a few groups:

- **CRUD** — create, query, update, and delete records, individually or in batches.

- **Associations** — has one, has many, belongs to, and many to many, plus preloading to load related data efficiently.
- **Schema management** — migrations driven by your model definitions, including automatic migration that creates tables and adds missing columns.
- **Lifecycle hooks** — your own code runs at fixed points, such as before a record is created or after it is found. Validation and timestamp bookkeeping usually live here.
- **Correctness** — transactions, including nested transactions and save points.
- **Lower-level access** — a SQL builder, subqueries, raw SQL execution, and prepared statement mode.
- **Diagnostics** — logging of the SQL GORM generates, which is invaluable while learning.

Two pieces underpin everything that follows. The first is the *gorm.DB handle returned by gorm.Open: it is what you pass around and call methods on. The second is error handling. GORM both returns errors from its operations and records them in db.Error, and checking them correctly is what separates code that fails loudly from code that fails silently.

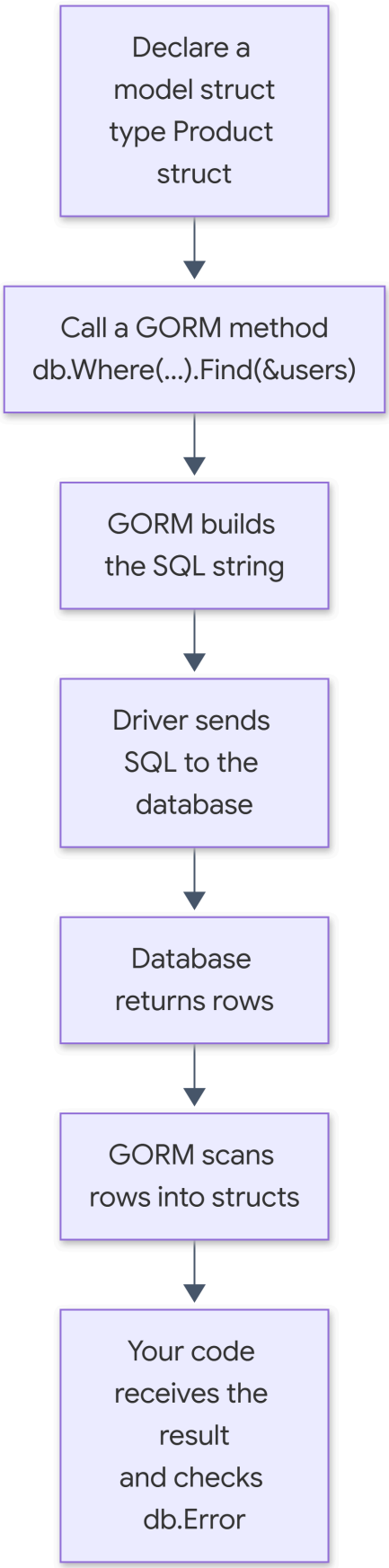

Declare a model struct type Product struct
Call a GORM method db.Where(...).Find(&users)
GORM builds the SQL string
Driver sends SQL to the database
Database returns rows
GORM scans rows into structs
Your code receives the result and checks db.Error

Fig 1.2 Lifecycle of a single GORM query from struct declaration through to scanned results

The diagram follows one query from end to end. It starts with the model struct you declared, then a call such as db.Where(...).Find(&users). GORM assembles the SQL, the driver transmits it, and the database returns matching rows. GORM scans those rows into the slice you passed, and your code receives the result together with db.Error to check. Every query in this book follows this path; the later chapters differ only in which stage they examine.

Traditional API and Generics API

GORM offers two ways to write the same operations, and you will meet both in real codebases.

The traditional API is method-based and centers on the *gorm.DB handle. Methods are chained, and the final method executes:

```
package main

import (
    "gorm.io/driver/sqlite"
```

```
    "gorm.io/gorm"
)

type Product struct {
    gorm.Model
    Code  string
    Price uint
}

func main() {
    db, err := gorm.Open(sqlite.Open("test.db"),
&gorm.Config{})
    if err != nil {
        panic("failed to connect database")
    }

    db.AutoMigrate(&Product{})

    // Create
    db.Create(&Product{Code: "D42", Price: 100})

    // Read
    var product Product
    db.First(&product, "code = ?", "D42")

    // Update
    db.Model(&product).Update("Price", 200)

    // Delete
```

```
    db.Delete(&product, 1)
}
```

The generics API, added in version 1.30.0, wraps the same engine in type-safe functions. You name the type once and the compiler knows what comes back, so there is no destination pointer to pass and no ambiguity about the result:

```
ctx := context.Background()

// Create
err = gorm.G[Product](db).Create(ctx, &Product{Code:
"D42", Price: 100})

// Read
product, err := gorm.G[Product](db).Where("code = ?",
"D42").First(ctx)

// Update
err = gorm.G[Product](db).Where("id = ?",
product.ID).Update(ctx, "Price", 200)

// Delete
err = gorm.G[Product](db).Where("id = ?",
product.ID).Delete(ctx)
```

The two styles are interchangeable and mix freely, because the generics API is a thin layer over the same implementation. The generics form catches more mistakes at compile time and returns errors directly rather than only recording them in db.Error. The traditional form carries more examples, blog posts, and answered questions behind it. Either is a sound starting point; this book uses the traditional API for most examples and notes the generics equivalent where the difference matters.

What GORM does not do for you

Seeing the tool's boundary matters as much as seeing its features, because the gap between what an ORM handles and what a production system needs is where most real problems live.

GORM generates SQL but never judges whether it is fast. A query that returns instantly on ten rows can take minutes on ten million, and GORM will not warn you. Reading the generated SQL and reasoning about the execution plan stays your job.

GORM can create and alter tables through automatic migration, but running that against a production database is a decision rather than a default. It is convenient in development and risky in deployment, where a dropped or narrowed column can destroy data.

GORM manages a connection pool, but the right settings depend on your database server, your traffic, and your latency. The defaults are a starting point, not an answer.

GORM provides transactions but cannot decide where your transaction boundaries belong. Only you know which operations must succeed or fail together.

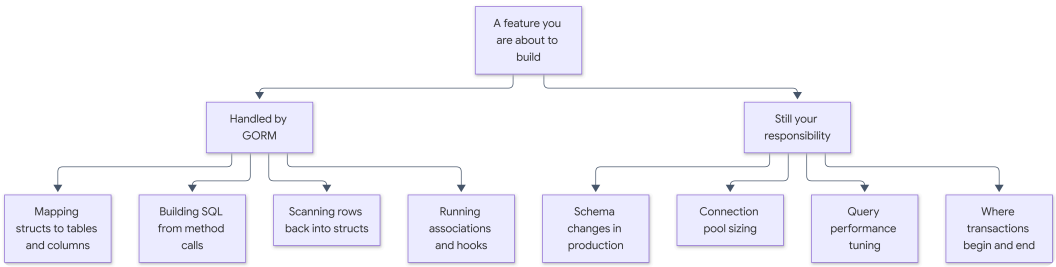


Fig 1.3 Division of responsibility between GORM and your application code

The diagram splits building a feature into two sides. On one, GORM handles the mechanical mapping: structs to tables and columns, method calls to SQL, rows back to

structs, along with the association and hook machinery. On the other, the decisions remain yours: how the schema changes in production, how the pool is sized, whether a query is fast enough, and where a transaction begins and ends. As you read on, it helps to keep asking which side each topic falls on.

Summary

An ORM maps Go structs onto relational tables and translates method calls into SQL in both directions, shielding you from the object-relational mismatch that makes hand-written data access tedious. GORM supplies CRUD operations, associations, migrations, hooks, transactions, a SQL builder, and logging, all reached through a single *gorm.DB handle — and it offers two API styles over that one implementation, the traditional chained method API and the type-safe generics API from version 1.30.0, so the choice between them is one of preference rather than capability.

The boundary matters more than the feature list. GORM generates SQL without guaranteeing it is fast, migrates

schemas you should not let run unattended in production, pools connections it cannot size for you, and provides transactions without knowing where yours belong. Holding on to that division of responsibility is what will keep the code you build with it healthy.

The next chapter takes the practical first step — installing GORM, opening a connection, and getting a program running — before models and conventions, which the rest of the book is built on.

Chapter 2: Setup

Nothing about mapping structs to tables matters until a connection opens. This chapter turns an empty directory into a Go module with GORM installed, a driver chosen, and a first connection verified. It then takes apart what you pass to gorm.Open, what the handle it returns manages underneath, and the few pool settings worth deciding on early. By the end you will have a running program — checked into this book's repository at example/setup/ — that creates a table, writes a row, and reads it back.

Installing GORM and a driver

GORM is distributed as two modules, and you install them separately. The first, gorm.io/gorm, is the ORM itself: models, queries, hooks, transactions, and the SQL builder. It contains no knowledge of any particular database. The second is a driver package, which teaches the core how to talk to one specific database and plugs it into a database/sql driver underneath.

Start a module and pull both in:

```
go mod init example.com/bookstore
go get -u gorm.io/gorm
go get -u gorm.io/driver/sqlite
```

`go mod init` runs once per project and its argument is a name you choose; the `example.com/` style is only a convention that avoids collisions if the module is ever published. `go get -u` downloads the latest release and records it in `go.mod`, with `go.sum` pinning the exact content afterward. Running `go list -m gorm.io/gorm` prints the version you actually have.

Pick the driver for the database you run. Each is a separate module, so install only the one you need:

Database	Driver module	Built on
SQLite	gorm.io/driver/sqlite	github.com/mattn/go-sqlite3 (requires CGO)
MySQL and compatible servers such as TiDB	gorm.io/driver/mysql	github.com/go-sql-driver/mysql
PostgreSQL	gorm.io/driver/postgres	github.com/jackc/pgx

The examples in this chapter were checked with Go 1.25 against gorm.io/gorm v1.31.2 and gorm.io/driver/sqlite v1.6.0.

Run go list -m all | findstr gorm — or grep on Linux and macOS — if you want to compare versions.

CGO and the SQLite driver

The SQLite driver deserves its own warning because it is the one almost every reader starts with, and it is the only one of the three with a build requirement beyond Go itself.

gorm.io/driver/sqlite is backed by github.com/mattn/go-sqlite3, which embeds the SQLite C library. Compiling it needs CGO: CGO_ENABLED=1 — the default whenever a C compiler is found — plus a compiler on your PATH. That is gcc on Linux, the Xcode command line tools on macOS, and MinGW-w64 or an equivalent GCC distribution on Windows (WSL also works, since the standard Linux toolchain includes everything needed).

The failure mode is worth recognizing, because it looks like a success first. Without a compiler the program still compiles — go-sqlite3 ships a stub that satisfies the build — and then refuses to run:

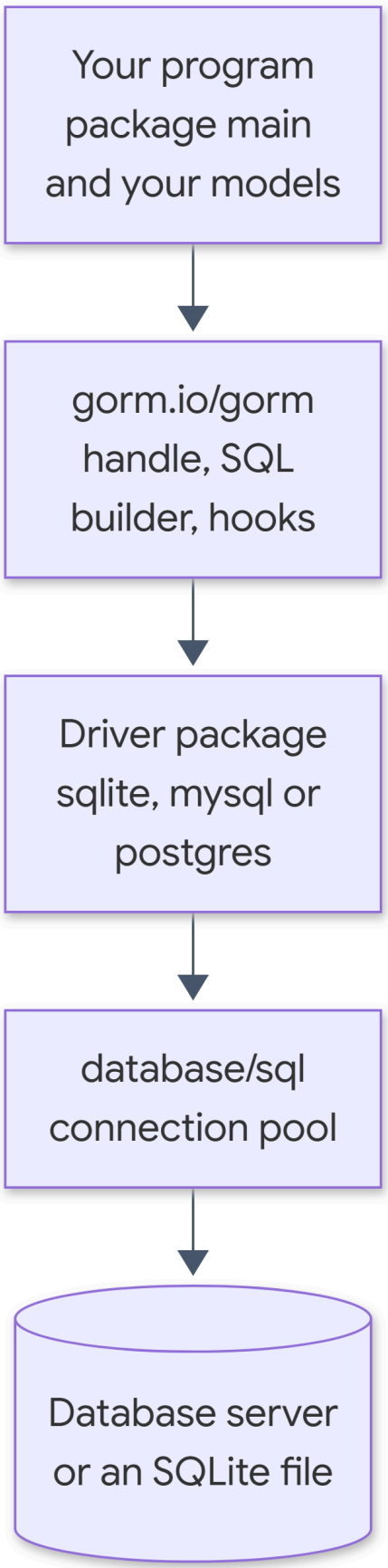
```
failed to connect database: Binary was compiled with
'CGO_ENABLED=0', go-sqlite3 requires cgo to work. This is
```

```
a stub
```

If you cannot install a C compiler, the official documentation lists pure-Go replacements that need nothing beyond the Go toolchain: github.com/glebarez/sqlite and github.com/libtnb/sqlite. They present the same package name and the same sqlite.Open function, so only the import line changes:

```
import "github.com/glebarez/sqlite" // pure Go — no C compiler required
```

The examples in this book use gorm.io/driver/sqlite. Keep the swap above in mind if you hit the stub error, and the rest of the code stays exactly as written.


Your program
package main
and your models
gorm.io/gorm
handle, SQL
builder, hooks
Driver package
sqlite, mysql or
postgres
database/sql
connection pool
Database server
or an SQLite file

Installed GORM stack from your program down to the database server

The diagram shows where the pieces you just installed sit relative to one another. Your program holds the *gorm.DB handle and calls methods on it; gorm.io/gorm builds the SQL; the driver package contributes the dialect — how this particular database names identifiers, what its types are — and opens the right kind of connection; database/sql owns the actual connections in a pool; and the server, or an SQLite file, answers at the bottom. Your code touches the top layer and the data that comes back up it, while everything in between is the plumbing this chapter is unpacking.

Connecting to SQLite, MySQL, and PostgreSQL

Every connection in GORM begins with gorm.Open, whose signature is:

```
func Open(dialector Dialector, opts ...Option) (db *DB,
err error)
```

The first argument is a *dialector*: the driver's Open function takes a DSN string and returns an object describing

how to reach that database. The second argument holds options, and in practice is almost always &gorm.Config{}. The call returns the handle you will pass around your program together with an error.

It does more than record the DSN. gorm.Open initializes the driver and pings the database by default, so a wrong host, a missing database, or a refused connection surfaces here rather than at your first query. The pattern is therefore fixed for the rest of the book:

```
db, err := gorm.Open(sqlite.Open("gorm.db"),
&gorm.Config{})
if err != nil {
    log.Fatal("failed to connect database: ", err)
}
```

SQLite

For SQLite the DSN is a file path, and nothing else is required:

```
db, err := gorm.Open(sqlite.Open("gorm.db"),
&gorm.Config{})
```

The file is created on first write if it does not exist, and it appears in the working directory of the process. For tests and experiments you can skip the file entirely with file::memory:?cache=shared, which keeps the database in system memory.

That shared form matters. A bare :memory: database is private to the connection that opened it, and a pooled application opens more than one connection — the symptom is a bewildering “no such table” error that appears only under load. The cache=shared form gives every connection in the process one shared in-memory database, which is what you want.

MySQL

MySQL takes a DSN in the user:pass@tcp(host:port)/dbname?params form understood by go-sql-driver/mysql:

```
dsn := "user:pass@tcp(127.0.0.1:3306)/gorm?
charset=utf8mb4&parseTime=True&loc=Local"
db, err := gorm.Open(mysql.Open(dsn), &gorm.Config{})
```

Two of those parameters are not optional in practice. parseTime=True is required for time.Time fields to scan correctly, and charset=utf8mb4 — not charset=utf8 — is what gives you full UTF-8, including four-byte characters such as emoji. The driver's documentation lists the rest of the parameters.

PostgreSQL

PostgreSQL uses a space-separated list of key=value pairs:

```
dsn := "host=localhost user=gorm password=gorm dbname=gorm
port=5432 sslmode=disable TimeZone=UTC"
db, err := gorm.Open(postgres.Open(dsn), &gorm.Config{})
```

sslmode=disable is acceptable against a local development server; anything reaching a real server should verify TLS rather than disable it. The PostgreSQL driver wraps pgx and enables a prepared statement cache by default. If something between your application and the database does not tolerate implicitly prepared statements, the driver can fall back to the simple query protocol:

```
db, err := gorm.Open(postgres.New(postgres.Config{
    DSN:                  dsn,
    PreferSimpleProtocol: true,
}), &gorm.Config{})
```

Keeping credentials out of the source

Every DSN above contains a password, and every one of them ends up committed if you hard-code it. Read it from the environment instead:

```
dsn := os.Getenv("DATABASE_DSN")
if dsn == "" {
    log.Fatal("DATABASE_DSN is not set")
}
```

The same applies to real credentials of any kind: the source tree is not a secret store, and the DSN belongs with the rest of the deployment configuration.

One more door into the same place: if your program already owns a *sql.DB — opened elsewhere, or shared with code that does not use GORM — hand it to the driver rather than letting GORM open its own:

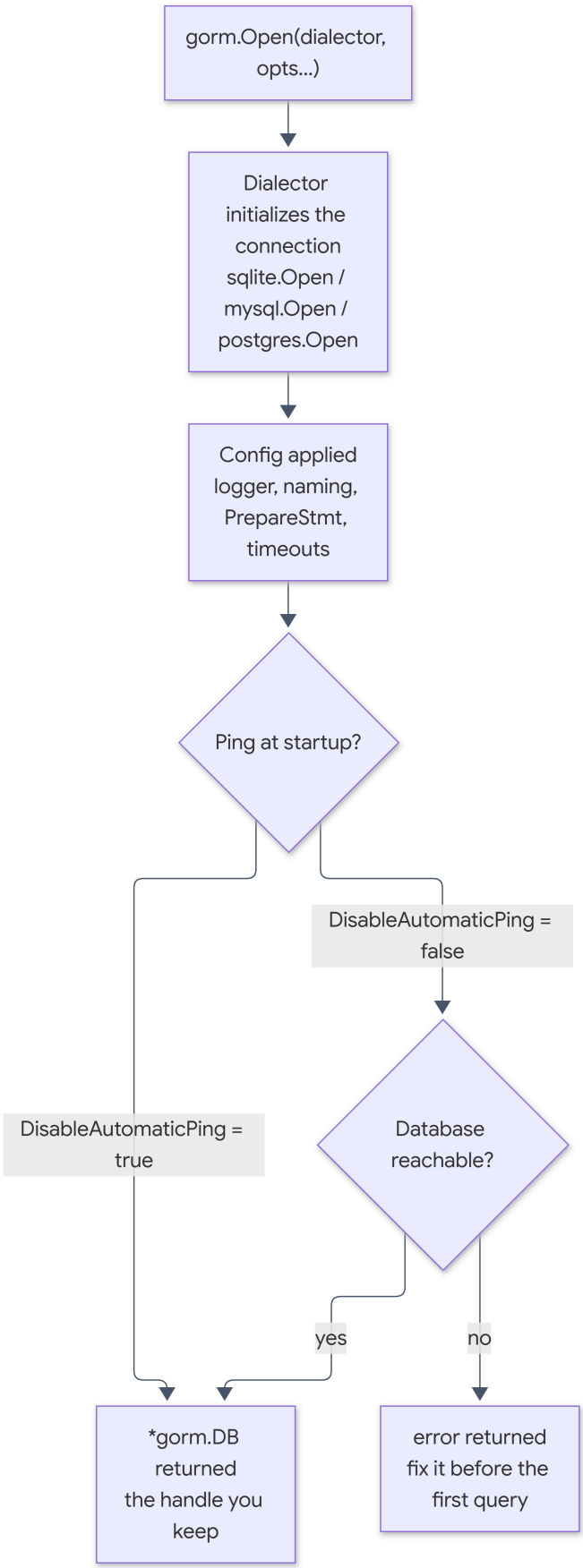
```
sqlDB, err := sql.Open("mysql", dsn)
if err != nil {
    log.Fatal(err)
}

db, err := gorm.Open(mysql.New(mysql.Config{Conn: sqlDB}),
&gorm.Config{})
```

The pool that connection already carries is the pool GORM will use, which is the subject of the section after next.

gorm.Config and connection options

The second argument to gorm.Open is where the behavior of the handle is decided. Its effects start at one moment and then persist: options are applied while gorm.Open initializes the connection, copied into the handle, and inherited by everything chained from it.


gorm.Open(dialector, opts...)
Dialector initializes the connection sqlite.Open / mysql.Open / postgres.Open
Config applied logger, naming, PrepareStmt, timeouts
Ping at startup?
DisableAutomaticPing = false
DisableAutomaticPing = true
Database reachable?
yes
no
*gorm.DB returned the handle you keep
error returned fix it before the first query

Steps inside gorm.Open from dialector and config to the returned handle or error

The diagram follows a single call to gorm.Open. The dialector establishes the connection and teaches GORM the dialect, the config is applied on top of that initialization, and then — unless you opted out with DisableAutomaticPing — the database is pinged. A reachable database gives you the *gorm.DB handle to keep; an unreachable one comes back as an error, before any query of yours has run. This ordering is why a configuration mistake or a dead server both show up at the same place: the gorm.Open call you already check.

For a first program the right config is the empty one, because the defaults are deliberately sane:

```
db, err := gorm.Open(sqlite.Open("gorm.db"),
&gorm.Config{})
```

The options you are most likely to reach for first:

Option	What it changes
SkipDefaultTransaction	Every single create, update, and delete runs inside its own transaction by default; set this

Option	What it changes
	to true to send them without one.
NamingStrategy	Table and column naming: prefixes, singular table names, name replacement. Chapter 3.
Logger	Replaces the default logger. Detailed below and in Chapter 16.
NowFunc	Supplies the timestamps GORM writes; it defaults to time.Now().Local().
DryRun	Builds SQL without executing it, for inspecting or testing what a chain would generate. Chapter 14.
PrepareStmt	Parses each statement once and caches the prepared form for reuse. Chapter 16.
DisableAutomaticPing	Skips the startup ping described above.
AllowGlobalUpdate	Permits an update or delete with no condition, which GORM otherwise refuses outright. Chapter 6.

Option	What it changes
DisableNestedTransaction	Turns off save points for transactions started inside a transaction. Chapter 12.
DisableForeignKeyConstraintWhenMigrating	AutoMigrate then creates tables without foreign key constraints. Chapter 10.

The struct carries more fields than this table — batch sizing, error translation, context timeouts among them — but these are the ones that surface during setup.

The Logger option is the one to change first while you are learning. The default logger writes to standard output at the Warn level, which means you see errors and slow statements — anything taking 200 ms or more — and nothing else. That is the right volume for a production service and the wrong one for someone reading GORM for the first time, so turn every statement on:

```
import "gorm.io/gorm/logger"

db, err := gorm.Open(sqlite.Open("gorm.db"), &gorm.Config{
```

```
    Logger: logger.Default.LogMode(logger.Info),
})
```

LogMode accepts Silent, Error, Warn, or Info and returns a new logger, leaving the original untouched, so the same default can be re-leveled per handle. You can also build your own with logger.New, pointing it at any io.Writer — or at log/slog through logger.NewSlogLogger — and Chapter 16 goes further into configuring it. Until then, logger.Info is the setting that makes the rest of this book observable:

```
db, err := gorm.Open(sqlite.Open("gorm.db"), &gorm.Config{
    Logger: logger.Default.LogMode(logger.Info), // drop back to Warn once the queries look right
    PrepareStmt:                true,
})
```

Config is settled once, at gorm.Open. When a single query later needs different treatment than the handle it came from — a longer timeout, a disabled prepared statement, a dry run — that is done with Session rather than by reopening the database, and Chapter 14 covers it.

Connection pools and database/sql settings

GORM does not maintain connections of its own. Underneath every handle sits a database/sql pool, and that pool is where the limits on concurrency actually live. GORM's own documentation is explicit about this: pooling is database/sql's job.

You reach it through the handle:

```
sqlDB, err := db.DB()
if err != nil {
    log.Fatal("failed to get database handle: ", err)
}
defer sqlDB.Close()
```

db.DB() returns the *sql.DB the handle was built on, so anything you set on it applies to every query GORM sends from then on. The defer matters too: closing sqlDB at shutdown drains the pool, and it is how connections are released when the process exits.

Four settings are worth knowing, and three of them have defaults you would not choose yourself:

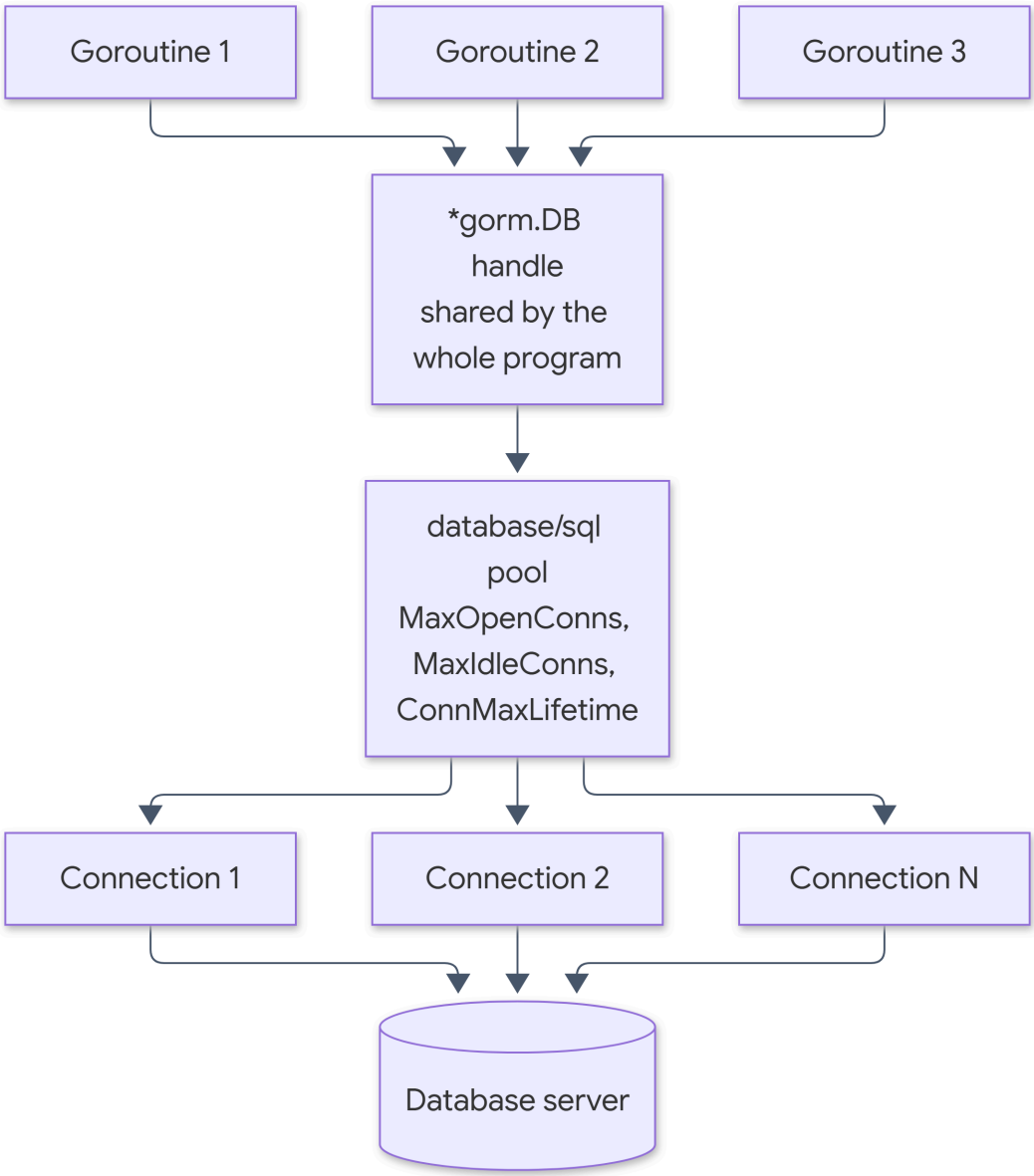
Setting	Default	Meaning
SetMaxOpenConns(n)	0 — unlimited	The most connections that may be open at once.
SetMaxIdleConns(n)	2	Connections kept open and ready for immediate reuse.
SetConnMaxLifetime(d)	0 — no limit	The longest a connection may be reused at all.
SetConnMaxIdleTime(d)	0 — no limit	The longest a connection may sit idle before being closed (Go 1.15+).

The default of two idle connections is a deliberate baseline rather than an oversight — the standard library keeps the pool small until told otherwise — but an unlimited number of open connections is not a setting, it is the absence of one. A reasonable starting point for a single service instance:

```
sqlDB.SetMaxOpenConns(25)
sqlDB.SetMaxIdleConns(10)
```

```
sqlDB.SetConnMaxLifetime(time.Hour)
```

Treat those numbers as a beginning to measure from, not an answer. MaxOpenConns must stay below the connection limit of the database server once every replica and instance of your service is counted, and below that ceiling, more headroom means queries wait less. Set it too low and requests queue behind the pool, showing up as latency spikes; set it too high and the server spends its time and memory on connections instead of queries. ConnMaxLifetime exists to recycle connections politely — around load balancers, firewall idling, and server restarts — rather than to fix a specific failure.



Goroutines sharing one handle through the database/sql connection pool

The diagram shows why these settings are global rather than per query. Any number of goroutines share the one *gorm.DB handle — it is safe to pass around — and every call descends into the same database/sql pool, which lends out

a physical connection for the duration of the call and takes it back afterward. The four settings sit on that middle layer, sizing how many connections may exist and how long each may be kept, so they shape the behavior of every query at once.

SQLite needs one caveat here. A file-backed SQLite database allows a single writer at a time, so pooled connections issuing writes concurrently will collide and report “database is locked”. Keep MaxOpenConns low — single digits — for anything SQLite-backed, or serialize writes yourself. When that constraint starts to hurt, it is the signal that the project has outgrown SQLite and belongs on MySQL or PostgreSQL.

A first working program

Everything in this chapter now assembles into one program. The complete source is checked in at example/setup/ as its own module, so you can run it directly — the first go run downloads the modules it needs:

```
cd example/setup
```

```
go run .
```

If you see the CGO stub error from earlier in the chapter, apply the fix from that section first. The program itself:

```
package main

import (
    "log"

    "gorm.io/driver/sqlite"
    "gorm.io/gorm"
    "gorm.io/gorm/logger"
)

type Product struct {
    gorm.Model
    Code  string
    Price uint
}

func main() {
    db, err := gorm.Open(sqlite.Open("test.db"),
&gorm.Config{
        Logger: logger.Default.LogMode(logger.Info),
    })
    if err != nil {
        log.Fatal("failed to connect database: ", err)
    }
```

```
    // the connection pool lives underneath the handle -
close it when main returns
    sqlDB, err := db.DB()
    if err != nil {
        log.Fatal("failed to get database handle: ", err)
    }
    defer sqlDB.Close()

    if err := db.AutoMigrate(&Product{}); err != nil {
        log.Fatal("failed to migrate database: ", err)
    }

    if err := db.Create(&Product{Code: "D42", Price:
100}).Error; err != nil {
        log.Fatal("failed to create product: ", err)
    }

    var product Product
    if err := db.First(&product, "code = ?", "D42").Error;
err != nil {
        log.Fatal("failed to find product: ", err)
    }

    log.Printf("found product: code=%s price=%d",
product.Code, product.Price)
}
```

With logger.Info enabled, the run prints the SQL behind each step:

```
2026/10/22 19:38:33 .../example/setup/main.go:32 [6.255ms]
[rows:0] CREATE TABLE `products` (`id` integer PRIMARY KEY
AUTOINCREMENT,`created_at` datetime,`updated_at`
datetime,`deleted_at` datetime,`code` text,`price`
integer)
2026/10/22 19:38:33 .../example/setup/main.go:36 [4.261ms]
[rows:1] INSERT INTO `products`
(`created_at`,`updated_at`,`deleted_at`,`code`,`price`)
VALUES ("2026-10-22 19:38:33.945","2026-10-22
19:38:33.945",NULL,"D42",100) RETURNING `id`
2026/10/22 19:38:33 .../example/setup/main.go:41 [0.522ms]
[rows:1] SELECT * FROM `products` WHERE code = "D42" AND
`products`.`deleted_at` IS NULL ORDER BY `products`.`id`
LIMIT 1
2026/10/22 19:38:33 found product: code=D42 price=100
```

(The records wrap onto two lines in a terminal, with the file path and elapsed time leading the statement; they are joined here, colour codes stripped, absolute paths shortened, and timings trimmed. AutoMigrate's catalog checks and its CREATE INDEX line are omitted. The line numbers match example/setup/main.go.)

Read the output top to bottom and the whole chapter is on display. AutoMigrate created test.db's products table from the struct — you can see the columns it derived, and

that the soft-delete column deleted_at is already there. The insert wrote one row and used RETURNING id to fetch the primary key back in the same statement. The query then shows two habits GORM takes on without being asked: it quoted the column, and it added deleted_at IS NULL — soft delete is active from your very first query, which Chapter 7 explains — while First contributed the ORDER BY and LIMIT 1 that make "first" mean first.

Two details in the code itself deserve naming. db.Create and db.First are chained calls, so their result is a *gorm.DB and the error arrives through .Error; gorm.Open and db.AutoMigrate, by contrast, return an error directly. Both styles are GORM's, and mixing the checks is not a mistake — it is the API telling you which shape each call has.

Run the program a second time and a second row appears: nothing in this model yet says a Code must be unique, so D42 is inserted again, and First still returns the oldest row. That is the chapter's honest ending — the connection works, and the modeling rules have not been written yet. Delete example/setup/test.db to start clean, because declaring those rules is what Chapter 3 does.

Summary

Setup turns out to be four decisions rather than one. You split GORM into its two modules, the pure-Go core and a driver package, and accepted whatever that driver brought with it — for SQLite, a C toolchain, unless you swap in a pure-Go driver. You built a DSN per database and learned the parameter each of them cannot do without: `parseTime` for MySQL, `sslmode` for PostgreSQL, `cache=shared` for an in-memory SQLite that survives pooling. And you found that `gorm.Open` does not merely record that DSN — it initializes the driver, applies your config, and pings, so a bad connection fails at the one line you already check.

Beneath the handle, `database/sql` runs the pool: two idle connections and no open limit until you say otherwise, reachable through `db.DB()` and closed with a `defer`. Sizing it is a measurement, not a setting — with one caveat for SQLite, whose single writer makes a large pool a liability rather than a feature. The config you inherited is quiet by default and was turned up to `Info` so the SQL GORM generates stays visible while you learn.

The next chapter is where the real mapping begins: what your struct declares becomes the table this program just created, decided by field names, tags, and the conventions GORM applies whether you asked for them or not.

Back Cover